Project Concord Comprehensive Master Report — v2.0

Veridian Unified Data Platform | Document Reference: VCH-PC-2026-FINAL-REPORT

Defense Date: Wednesday, 20 August 2026 @ 12:00 PM | Lead Architect: Samuel Enyi

PROJECT CONCORD: VERIDIAN UNIFIED DATA PLATFORM

Comprehensive Project Engineering & Architecture Report

Document Reference: VCH-PC-2026-FINAL-REPORT
Version: 2.0 (Corrected & Final Submission Edition)
Target Client: Office of the Group CEO (Adaeze Okonji-Bello) & Office of the CDTO (Tobenna Iweka)
Project Defense Date: Wednesday, 20 August 2026 @ 12:00 PM
Lead SQL Architect & Data Engineer: Samuel Enyi
Source Cohort: Veracity Vanguard

TABLE OF CONTENTS

PROJECT CONCORD: VERIDIAN CONSOLIDATED HOLDINGS

Business Requirements Document (BRD) — Veridian Unified Data Platform

Document Reference: VCH-PC-2026-BRD
Version: 1.0 (Final Architecture Review Edition)
Target Defense Date: Friday, 14 August 2026 @ 9:00 AM Prompt
Client: Office of the Group CEO & CDTO, Veridian Consolidated Holdings
Lead Author: Israel John (Business Analyst Lead) & Samuel Enyi (Lead Database Architect)

---

1. Executive Summary & Project Mandate

Veridian Consolidated Holdings (VCH) is a diversified West African conglomerate operating across 5 core operating divisions: **AgriCore, Meridian Retail, Concord Logistics, Veridian Financial Services (VFS), and Veridian Properties**. The group generates ₦340 Billion in annual revenue, employs ~14,600 staff, and serves 2.1 Million customers across Nigeria, Ghana, and Kenya.

The Business Mandate

Twenty years of organic growth and acquisitions have left VCH with 5 isolated technology stacks. This fragmentation costs VCH **4% to 6% of annual revenue (approx. ₦13.6B to ₦20.4B each year)** in stockouts, idle assets, delayed financial closing (21 calendar days), and missed cross-selling opportunities.

Project Concord mandates the design and implementation of a single, well-governed operational database—the **Veridian Core Platform**—built around a **Core Services Hub (6 shared entities)** surrounded by **5 divisional modules**, linked via scheduled ELT into **Google BigQuery** for executive analytics.

---

2. Business Objectives & Success Criteria

The platform must explicitly solve the **4 Critical Real-World Stakeholder Scenarios**:

```
+--------------------------------------------------------------------------------+
|                           THE 4 STAKEHOLDER SUCCESS TARGETS                    |
+--------------------------------------------------------------------------------+
| 1. STORE MANAGER (Ngozi): Real-time alert on AgriCore harvest shortfalls to prevent|
|    stockouts at Meridian Mart supermarkets 24-48 hours before shelves go empty.   |
| 2. LOAN OFFICER (Chinedu): Cross-divisional view of smallholder farmer (Ibrahim's)  |
|    6-year AgriCore supply record to approve working capital loans with lower risk. |
| 3. LOGISTICS DISPATCHER (Funmi): Real-time visibility into Veridian Properties    |
|    warehouse leases to prevent double-booking depot bays and shipping delays.     |
| 4. GROUP CEO (Adaeze Okonji-Bello): Same-day consolidated revenue & P&L view across |
|    all 5 divisions (reducing month-end financial close from 21 days to < 24 hours).|
+--------------------------------------------------------------------------------+
```

---

3. Scope Boundaries & Constraints (Avoiding Past Failures)

To avoid the failures of VCH's 2019 attempt (monolithic scope creep, 0 tables shipped) and 2022 attempt (200+ tables, dual MySQL/Oracle, running OLAP on live OLTP):

- **In Scope**:
- **40 to 55 tables** maximum.
- PostgreSQL hosted on Supabase (OLTP System of Record).
- Google BigQuery (OLAP Analytical Warehouse).
- Row Level Security (RLS) policies and Customer Consent Flags.
- 10,000+ synthetic records covering realistic cross-divisional customer/farmer overlap.
- **Out of Scope**:

- Replacing VFS's internal core banking engine. VFS is treated as an **external read-restricted source**.
- Building mobile or web application front-ends.
- Dual-database platform implementations (MySQL/Oracle).

---

4. Divisional Business Rules & Schema Constraints

Below are the exact business constraints that **Samuel Enyi** must code into the SQL DDL schema (`01_schema_ddl.sql`):

4.1 Core Services Hub Rules (Shared Entities)

1. **`customers`** : Every individual customer must have a unique `customer_id` (e.g. `CORE_CUST_XXXXXX`), valid ISO country code (`NG` , `GH` , `KE`), and explicit consent boolean flags (`consent_retail_loyalty` , `consent_vfs_credit_sharing`).
2. **`employees`** : Staff members belong to exactly one division (`division_id`), with employment status `ACTIVE` , `ON_LEAVE` , or `TERMINATED` .
3. **`suppliers_vendors`** : Unified vendor master list. Every AgriCore farmer is registered as a supplier (`supplier_type = 'FARMER'`).
4. **`locations_sites`** : Universal address and geocoordinate standard (`latitude` , `longitude`) for all stores, warehouses, farms, and properties.
5. **`product_service_catalogue`** : Shared master SKU list (`product_id` , `unit_of_measure`) shared between AgriCore harvest outputs and Meridian Retail inventory items.
6. **`financial_account_references`** : Read-only reference link (`account_ref_id` , `customer_id` , `account_type` , `status`) connecting customers to VFS accounts without exposing raw banking transactions to unprivileged roles.

4.2 Divisional Module Rules

- **Meridian Retail Module**:
- `stores` : Must reference a valid `location_id` in Core Hub.
- `pos_transactions` : `total_amount` must be greater than zero (`CHECK (total_amount > 0)`).
- `inventory_stock_levels` : `quantity_on_hand` cannot be negative (`CHECK (quantity_on_hand >= 0)`).
- **Concord Logistics Module**:
- `vehicles` : Must have a valid registration number and `capacity_kg > 0` .
- `shipments` : `destination_location_id` must differ from `origin_location_id` .
- `warehouses` : `capacity_units` must link to an active property lease in Veridian Properties.
- **Veridian Financial Services (VFS) Module**:
- `wallet_accounts` : `balance` cannot be negative unless an approved overdraft limit exists (`CHECK (balance >= overdraft_limit)`).
- `loans` : `principal_amount > 0` , `interest_rate > 0` . Statuses: `APPLIED` , `DISBURSED` , `ACTIVE` , `PAID_OFF` , `DEFAULTED` (NPL = payments overdue > 90 days).
- `kyc_records` : Tier 1, Tier 2, Tier 3 verification level strictly access-restricted under CBN microfinance guidelines.
- **AgriCore Module**:
- `farms` : Must link to a registered farmer in Core Hub (`supplier_id`). `size_hectares > 0` .
- `harvest_batches` : Must record `harvest_date` , `volume_kg` , and `field_agent_employee_id` .
- `processing_runs` : Must reference a valid `harvest_id` and assign a quality grade (`GRADE_A` , `GRADE_B` , `GRADE_C` , `REJECTED`).
- **Veridian Properties Module**:
- `leases` : `lease_end_date` must fall after `lease_start_date` (`CHECK (end_date > start_date)`).
- `monthly_rent` must be positive.

---

5. Regulatory & Security Compliance Requirements

1. Central Bank Microfinance Regulation

VFS banking records are subject to strict regulatory segregation. Non-banking staff (e.g. retail store managers, logistics dispatchers) **MUST NOT** be granted direct table read access to raw VFS wallet or loan transaction tables. Access is restricted using PostgreSQL **Row Level Security (RLS)**.

2. Nigeria Data Protection Act (NDPA) & Regional Privacy Laws

Customer data sharing across divisions requires explicit consent. The database must evaluate `consent_vfs_credit_sharing = TRUE` before exposing an AgriCore farmer's supply history to a VFS loan officer.

6. Document Sign-off & Handover

This BRD serves as the authoritative specification for **Samuel Enyi (Lead Database Architect)** to construct the `01_schema_ddl.sql` script and **Ruth Thompson** to generate synthetic CSV datasets.

- **Business Analyst Lead**: Israel John (Signed)
- **Lead Database Architect**: Samuel Enyi (Signed)
- **Data Engineering Lead**: Ruth Thompson (Signed)
- **BI Developers**: Christian Nwobodo & Richmond Akara-Nwaogu (Signed)

PROJECT CONCORD: VERIDIAN FINANCIAL SERVICES

Master Project Execution Plan: Pure SQL & Analytics Division

Document Version: 3.0 (Pure SQL vs. Analytics Division)
Target Defense Date: Friday, 14 August 2026 @ 9:00 AM Prompt
Division: Veridian Financial Services
Source Cohort: Veracity Vanguard
Lead SQL Architect & Data Engineer: Samuel Enyi

---

1. Executive Summary & Core Division Rule

To maximize efficiency and play to everyone's strengths:

- **SAMUEL ENYI** takes **100% FULL OWNERSHIP of ALL SQL & DATABASE ENGINEERING**. Samuel designs the ERD, writes all DDL table scripts, builds foreign key constraints, executes database seeding, creates analytical SQL views, implements security, and tunes query performance.
- **THE TEAM (Israel, Christian, Richmond, Ruth)** takes **100% FULL OWNERSHIP of BUSINESS ANALYTICS, BI DASHBOARDING, SYNTHETIC DATA GENERATION & DEFENSE PRESENTATION**.

```
+-------------------------------------------------------------------------------------+
|                         SAMUEL ENYI (SQL & Database Owner)                          |
| • 100% SQL DDL Scripts    • 100% Database Seeding     • 100% Analytical SQL Views   |
| • 100% ERD & Constraints • 100% Indexing & Security • 100% Database Administration|
+------------------------------------------+------------------------------------------+
                                           | (Exposes SQL Views / Connection String)
        +----------------------------------+----------------------------------+
        |                                  |                                  |
+-------v-------+                  +-------v-------+                  +-------v-------+
|  ISRAEL JOHN  |                  | CHRISTIAN &   |                  | RUTH THOMPSON |
| Business Logic|                  | RICHMOND      |                  | Synthetic Data|
| & Presentation|                  | BI Dashboards |                  | Mockaroo CSVs |
| Storyboarding |                  | (Power BI)    |                  | & QA Testing  |
+---------------+                  +---------------+                  +---------------+
```

---

2. Definitive Role Allocation Matrix

Team Member	Domain Ownership	Assigned Operational Title	Complete Responsibilities & Deliverables
Samuel Enyi	**100% SQL & Database**	**Lead Database Architect & SQL Engineer**	• **All Data Modeling**: Design 3NF OLTP Schema & Star/Snowflake OLAP ERD. • **All DDL Scripts**: Write `01_schema_ddl.sql` (Tables, PKs, FKs, Indexes, Constraints). • **All Data Ingestion**: Write `02_seed_data.sql` and run database population scripts. • **All SQL Views**: Build `03_analytics_views.sql` for BI tools to connect to. • **Database Security & Optimization**: Apply `04_security.sql` (PII masking) and `05_indexing.sql` (`EXPLAIN ANALYZE`).
Israel John	**100% Business & Project Ops**	**Project Manager & Business Analyst**	• **Business Rules**: Author `Project_Concord_BRD.md` (Loan terms, interest rates, overdraft rules). • **KPI Definitions**: Define exact metrics for BI developers (e.g. NPL ratio formula, churn logic). • **Project Management**: Track daily sprint progress and team deadlines. • **Defense Deck**: Lead presentation slide deck creation & co-anchor defense.
Christian Nwobodo	**100% BI Financial Analytics**	**Lead Financial BI Developer**	• **Financial Dashboarding**: Connect Power BI / Tableau / Excel to Samuel's SQL Views. • **Visual Metrics**: Build cards and charts for Revenue, Total Deposits, Interest Income, Net Margin. • **Formatting**: Customize theme colors, slicers, time filters, and layout. • **Presentation**: Present Financial Analytics live during defense.
Richmond Akara-Nwaogu	**100% BI Customer & Credit Analytics**	**Lead Credit & Risk BI Developer**	• **Risk Dashboarding**: Connect Power BI / Tableau / Excel to Samuel's SQL Views. • **Risk Metrics**: Build visual gauges and charts for Non-Performing Loans (NPL %), Delinquencies. • **Customer Analytics**: Visualize Credit Score distributions, active users, and high-risk flags. • **Presentation**: Present Customer & Credit Risk analytics live during defense.
Ruth Thompson	**100% Data Prep & QA Documentation**	**Data Generation & QA Lead**	• **Synthetic Data Prep**: Generate realistic financial CSV datasets (names, BVNs, balances, loans) using Mockaroo / Excel. • **Data Handover**: Hand formatted CSVs to Samuel for SQL seeding. • **Data Quality Assurance**: Test dashboard numbers against CSV inputs to verify accuracy. • **Documentation**: Compile the final **Data Dictionary** and documentation binder.

3. Clear Interface Contract Between Samuel & The Team

To ensure smooth collaboration:

```
[Ruth Thompson] -------- Generates CSVs (Mockaroo) ------> [Samuel Enyi]
                                                                |
                                                 Writes SQL DDL, Seeds Data,
                                                 & Creates SQL Views
                                                                |
[Christian & Richmond] <--- Connects Power BI to SQL Views -----+
```

1. **Input Contract**: Ruth generates raw financial CSV files and hands them to Samuel.
2. **Database Engine**: Samuel loads CSVs into the PostgreSQL/MySQL database and executes all DDL and View scripts.
3. **Output Contract**: Samuel provides Christian and Richmond with the Database Connection details (Host, Database Name, View Names, e.g., `v_financial_overview` , `v_credit_risk`).

4. **Dashboard Building**: Christian and Richmond build interactive Power BI / Tableau reports purely using visual drag-and-drop elements on top of Samuel's views.

4. 4-Week Roadmap

```
gantt
    title Detailed Execution Roadmap (SQL vs Analytics Split)
    dateFormat  YYYY-MM-DD

    section Phase 1: Requirements & BRD
    Business Requirements BRD (Israel)      :2026-07-21, 4d
    Shared Core Hub Keys Sync (Samuel)      :2026-07-24, 2d

    section Phase 2: Schema & Synthetic CSVs
    ERD & 3NF SQL Schema Setup (Samuel)     :2026-07-26, 4d
    Mockaroo Synthetic CSV Creation (Ruth)  :2026-07-27, 4d

    section Phase 3: SQL Seeding & View Building
    SQL Data Ingestion & Seeding (Samuel)   :2026-07-31, 3d
    Pre-Built Analytical SQL Views (Samuel) :2026-08-03, 3d
    Data Lineage & QA Review (Ruth)         :2026-08-05, 2d

    section Phase 4: BI Dashboards & Defense
    Power BI Financial Dashboard (Christian):2026-08-07, 4d
    Power BI Credit Risk Dashboard (Richmond):2026-08-07, 4d
    Slide Deck & Presentation Dry-Run (Israel):2026-08-11, 3d
    FINAL PROJECT DEFENSE                   :milestone, 2026-08-14, 1d
```

5. Defense Presentation Speaking Breakdown

Each team member will own their domain during the 15-minute August 14 Defense:

Time Slot	Presenter	Topic Covered
0:00 - 3:00 (3 mins)	**Israel John**	**Project Overview & Business Logic**: Introduction, Problem Statement, Financial Business Rules (BRD).
3:00 - 8:00 (5 mins)	**Samuel Enyi**	**Complete Database Architecture**: ERD Design, 3NF Normalization, DDL Scripts, Constraints, Foreign Keys, SQL Views, & Query Performance Tuning (`EXPLAIN ANALYZE`).
8:00 - 10:00 (2 mins)	**Ruth Thompson**	**Data Engineering & Integrity**: Synthetic Data Generation (15,000+ records), Data Dictionary, and QA Validation.
10:00 - 13:00 (3 mins)	**Christian Nwobodo**	**Financial BI Dashboard Demo**: Live presentation of Revenue, Total Deposits, Balances, and Net Margin.
13:00 - 15:00 (2 mins)	**Richmond Akara-Nwaogu**	**Credit Risk BI Dashboard Demo**: Live presentation of NPL %, Delinquencies, Credit Scores, and Risk Segmentation.

Architected and Structured by Samuel Enyi, Lead Database Architect & SQL Engineer.

PROJECT CONCORD: LEAD DATA ARCHITECT GUIDE

Samuel Enyi's Step-by-Step SQL Engineering & Database Release Roadmap

This guide defines the precise engineering execution steps for **Samuel Enyi** to build, secure, optimize, and hand over the **Veridian Unified Data Platform** database.

1. Technical Task Breakdown & Sequence

```
┌─────────────────────────────────────────────────────────┐
│  Step 1: Conceptual ERD (dbdiagram.io & Mermaid)        │
└─────────────────────────────────────────────────────────┘
               │
               ▼
┌─────────────────────────────────────────────────────────┐
│  Step 2: Core Services Hub Tables & Constraints (DDL)    │
└─────────────────────────────────────────────────────────┘
               │
               ▼
┌─────────────────────────────────────────────────────────┐
│  Step 3: Divisional Modules Tables & Constraints (DDL)  │
└─────────────────────────────────────────────────────────┘
               │
               ▼
┌─────────────────────────────────────────────────────────┐
│  Step 4: RLS Security Policies & Role Grants (Supabase) │
└─────────────────────────────────────────────────────────┘
               │
               ▼
┌─────────────────────────────────────────────────────────┐
│  Step 5: Load Seed Data (pgAdmin / DBeaver CSV Import)  │
└─────────────────────────────────────────────────────────┘
               │
               ▼
┌─────────────────────────────────────────────────────────┐
│  Step 6: Pre-build Analytical SQL Views for Power BI    │
└─────────────────────────────────────────────────────────┘
```

2. Step-by-Step Technical Guide

Step 1: Conceptual & Logical ERD Setup

- **Action**: Use dbdiagram.io.
- **Rule**: Map the **6 Core Services Hub entities** in the center and the **5 divisional modules** around it.
- **Goal**: Define primary keys and foreign key relationships with clear cardinalities before writing any DDL code.

Step 2: Implement Core Services Hub DDL

Create the shared foundation tables. Apply naming standards (snake_case, plural table names, singular `_id` keys):
- **`customers`** : Contains `customer_id` (UUID or SERIAL), `full_name` , `email` , `consent_retail_loyalty` (Boolean), `consent_vfs_credit_sharing` (Boolean).
- **`employees`** : Contains `employee_id` , `full_name` , `division_id` , `role_title` , `reports_to` (self-referential FK).
- **`suppliers_vendors`** : Unified registry. Farmers are flagged as `supplier_type = 'FARMER'` .
- **`locations_sites`** : Shared geocoordinates (`latitude DECIMAL(9,6)` , `longitude DECIMAL(9,6)`).
- **`product_service_catalogue`** : Shared SKUs (`product_id` , `product_name` , `unit_of_measure`).
- **`financial_account_references`** : References VFS account IDs (`account_ref_id` , `customer_id` , `account_type` , `status`) without storing private ledger columns.

Step 3: Implement Divisional Module Tables

Create specific tables for the divisions. Ensure strict constraints to maintain business logic integrity:
- **`pos_transactions`** : Add `CHECK (total_amount > 0)` .
- **`inventory_stock_levels`** : Add `CHECK (quantity_on_hand >= 0)` .
- **`loans`** : Add `CHECK (principal_amount > 0)` .
- **`leases`** : Add `CHECK (end_date > start_date)` .
- **`harvest_batches`** : Link back to Core Hub (`farm_id` , `product_id`).

Step 4: Write Row Level Security (RLS) Policies

Enforce security at the database engine level (PostgreSQL) rather than application code:
1. **Enable RLS**:
`sql ALTER TABLE veridian_financial_services.wallet_accounts ENABLE ROW LEVEL SECURITY; ALTER TABLE veridian_financial_services.loans ENABLE ROW LEVEL SECURITY;`
2. **Define Access Roles**:
`sql CREATE ROLE retail_staff; CREATE ROLE financial_staff;`
3. **Write Consent-based Policies**:
`sql -- Allow VFS staff to view AgriCore harvest details ONLY IF the farmer has consented: CREATE POLICY vfs_farmer_view_policy ON agricore.farms FOR SELECT TO financial_staff USING (EXISTS (SELECT 1 FROM core.customers c WHERE c.customer_id = farms.supplier_id AND c.consent_vfs_credit_sharing = TRUE));`

Step 5: Execute Synthetic Data Ingestion

- Coordinate with **Ruth Thompson** to get the generated CSV data files.
- Use pgAdmin Import/Export Tool or PostgreSQL `COPY` command to seed data.
- Ensure correct loading sequence: **Core Hub tables first**, followed by divisional transactional tables.

Step 6: Create Curated Analytical Views

Expose clean, pre-joined views so the BI developers don't have to write complex joins.

View 1: Ngozi's Store Stock & Supply Alert

```
CREATE VIEW v_agricore_retail_supply_alert AS
SELECT
    s.store_id,
    l.site_name AS store_location,
    p.product_name,
    isl.quantity_on_hand AS retail_stock_level,
    hb.volume_kg AS recent_harvest_volume,
    hb.harvest_date
FROM retail.stores s
JOIN core.locations_sites l ON s.location_id = l.location_id
JOIN retail.inventory_stock_levels isl ON s.store_id = isl.store_id
JOIN core.product_service_catalogue p ON isl.product_id = p.product_id
LEFT JOIN agricore.harvest_batches hb ON p.product_id = hb.product_id;
```

View 2: Chinedu's Farmer Credit Evaluation View

```
CREATE VIEW v_farmer_credit_evaluation AS
SELECT
    c.customer_id,
    c.full_name AS farmer_name,
    f.farm_id,
    f.primary_crop,
    SUM(hb.volume_kg) AS total_harvested_volume_kg,
    c.consent_vfs_credit_sharing
FROM core.customers c
JOIN agricore.farms f ON c.customer_id = f.supplier_id
JOIN agricore.harvest_batches hb ON f.farm_id = hb.farm_id
WHERE c.consent_vfs_credit_sharing = TRUE
GROUP BY c.customer_id, c.full_name, f.farm_id, f.primary_crop;
```

---

3. Database Connection Parameters (Handover Package)

Once database script is loaded and views are created, copy the connection details below and hand them over to **Christian** and **Richmond** to connect Power BI:

- **Host**: `db.supabase.co` (replace with your Supabase Host URL)
- **Port**: `5432`
- **Database**: `postgres`
- **Portals / Views to Connect**:
- `v_agricore_retail_supply_alert`
- `v_farmer_credit_evaluation`
- `v_warehouse_lease_utilization`
- `v_executive_consolidated_revenue`

PROJECT CONCORD: VERIDIAN UNIFIED DATA PLATFORM

Master Enterprise Data Dictionary (All 43 Entities)

Document Reference: VCH-PC-2026-DD
Version: 1.0 (Final Enterprise Release)
Author: Samuel Enyi (Lead Database Architect & SQL Engineer)
Database Engine: PostgreSQL 15 (Supabase Managed Engine)
Target Defense Date: Friday, 14 August 2026 @ 9:00 AM Prompt

1. Executive Overview & Schema Organization

The **Veridian Unified Data Platform** follows a **Core Hub & Divisional Modules** architecture. The platform consists of **6 distinct logical schemas** housing **43 primary operational entities** (and associated analytical views):

```
                         +---------------------------------+
                         |    CORE SERVICES HUB (core)     |
                         | • locations_sites               |
                         | • customers                     |
                         | • suppliers_vendors             |
                         | • employees                     |
                         | • product_service_catalogue     |
                         | • financial_account_references  |
                         +----------------+----------------+
                                          |
       +-------------------+--------------+-------------------+------------------+
       |                   |              |                   |                  |
  +----v----+         +----v----+    +----v----+         +----v----+        +----v----+
  | RETAIL  |         |   VFS   |    |AGRICORE |         |LOGISTICS|        |PROPERTIES|
  | (7 tbls)|         | (6 tbls)|    | (7 tbls)|         | (7 tbls)|        | (7 tbls)|
  +---------+         +---------+    +---------+         +---------+        +---------+
```

2. Core Services Hub (`core` Schema — 6 Shared Entities)

2.1 `core.locations_sites`

- **Purpose**: Single source of truth for all physical and operational sites across Nigeria (NG), Ghana (GH), and Kenya (KE).
- **Referenced By**: `retail.stores` , `agricore.farms` , `agricore.processing_runs` , `logistics.shipments` , `logistics.routes` , `logistics.warehouses` , `properties.properties` .

Column Name	Data Type	Constraints	Description / Business Rules
`location_id`	`VARCHAR(30)`	`PRIMARY KEY`	Unique location identifier (e.g., `LOC-NG-LOS-001`).
`site_name`	`VARCHAR(150)`	`NOT NULL`	Descriptive name of the site (e.g., `Ikeja Hypermarket Premises`).
`site_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	Type of facility: `'STORE'` , `'WAREHOUSE'` , `'FARM'` , `'PROPERTY'` , `'OFFICE'` .
`address`	`TEXT`	`NOT NULL`	Physical street address.
`city`	`VARCHAR(100)`	`NOT NULL`	City or municipality (e.g., `Lagos` , `Kano` , `Accra`).
`country_code`	`VARCHAR(5)`	`NOT NULL` , `CHECK`	ISO 2-letter country code: `'NG'` , `'GH'` , `'KE'` .
`latitude`	`DECIMAL(9,6)`	Nullable	Geolocation latitude coordinate.
`longitude`	`DECIMAL(9,6)`	Nullable	Geolocation longitude coordinate.
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Record creation timestamp.

2.2 `core.customers`

- **Purpose**: Unified identity master for all individual customers interacting with Meridian Retail and Veridian Financial Services.
- **Referenced By**: `retail.pos_transactions` , `retail.loyalty_accounts` , `vfs.wallet_accounts` , `vfs.loans` , `vfs.kyc_records` , `core.financial_account_references` .

Column Name	Data Type	Constraints	Description / Business Rules
`customer_id`	`VARCHAR(30)`	`PRIMARY KEY`	Global customer ID (e.g., `CUST-100452`).
`full_name`	`VARCHAR(150)`	`NOT NULL`	Legal full name of the individual customer.
`date_of_birth`	`DATE`	`NOT NULL`	Date of birth for KYC and age verification.
`primary_contact`	`VARCHAR(50)`	`NOT NULL`	Phone number or primary email address.
`registered_country`	`VARCHAR(5)`	`NOT NULL` , `CHECK`	Registration country: `'NG'` , `'GH'` , `'KE'` .
`consent_retail_loyalty`	`BOOLEAN`	`DEFAULT TRUE`	Consent flag for retail marketing & loyalty program.
`consent_vfs_credit_sharing`	`BOOLEAN`	`DEFAULT FALSE`	Consent flag permitting VFS credit assessment access.
`created_date`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Customer registration timestamp.

2.3 `core.suppliers_vendors`

- **Purpose**: Shared master registry for external suppliers, vendor partners, and AgriCore smallholder farmers.
- **Referenced By**: `agricore.farmers` , `agricore.farms` , `retail.supplier_deliveries` , `vfs.loans` .

Column Name	Data Type	Constraints	Description / Business Rules
`supplier_id`	`VARCHAR(30)`	`PRIMARY KEY`	Global supplier identifier (e.g., `SUPP-AGRI-0012`).
`legal_name`	`VARCHAR(150)`	`NOT NULL`	Legal business name or farmer full name.
`supplier_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	Classification: `'FARMER'` , `'WHOLESALE_VENDOR'` , `'SERVICE_PROVIDER'` .
`primary_division_id`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	Primary division relationship: `'AGRICORE'` , `'RETAIL'` , `'LOGISTICS'` , `'VFS'` , `'PROPERTIES'` .
`onboarding_date`	`DATE`	`NOT NULL`	Date supplier was onboarded.
`status`	`VARCHAR(30)`	`DEFAULT 'ACTIVE'` , `CHECK`	Account status: `'ACTIVE'` , `'SUSPENDED'` , `'INACTIVE'` .

2.4 `core.employees`

- **Purpose**: Group-wide employee registry across all 5 operating divisions and executive leadership.
- **Referenced By**: `retail.stores` (manager), `agricore.harvest_batches` (agent), `agricore.quality_grades` (inspector), `logistics.drivers` (driver employee).

Column Name	Data Type	Constraints	Description / Business Rules
`employee_id`	`VARCHAR(30)`	`PRIMARY KEY`	Unique staff ID (e.g., `EMP-10023`).
`full_name`	`VARCHAR(150)`	`NOT NULL`	Employee full name.
`division_id`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	Operating division: `'RETAIL'` , `'LOGISTICS'` , `'AGRICORE'` , `'VFS'` , `'PROPERTIES'` , `'EXECUTIVE'` .
`role_title`	`VARCHAR(100)`	`NOT NULL`	Job designation (e.g., `Store Manager` , `Field Agent`).
`employment_status`	`VARCHAR(30)`	`DEFAULT 'ACTIVE'` , `CHECK`	Status: `'ACTIVE'` , `'ON_LEAVE'` , `'TERMINATED'` .
`hire_date`	`DATE`	`NOT NULL`	Official date of employment.
`reports_to`	`VARCHAR(30)`	`FK -> core.employees`	Manager's employee ID (Self-referential hierarchy).

2.5 `core.product_service_catalogue`

- **Purpose**: Master product and service master list sold through Meridian Retail or produced by AgriCore.
- **Referenced By**: `retail.transaction_line_items`, `retail.inventory_stock_levels`, `retail.promotions`, `agricore.harvest_batches`.

Column Name	Data Type	Constraints	Description / Business Rules
`product_id`	`VARCHAR(30)`	`PRIMARY KEY`	Unique product SKU code (e.g., `PROD-GRN-001`).
`product_name`	`VARCHAR(150)`	`NOT NULL`	Commercial name (e.g., `Yellow Maize Grain 50kg`).
`category`	`VARCHAR(50)`	`NOT NULL`, `CHECK`	Category: `'FRESH_PRODUCE'`, `'GRAINS'`, `'PACKAGED_GOODS'`, `'BEVERAGES'`, `'SERVICES'`.
`unit_of_measure`	`VARCHAR(20)`	`NOT NULL`, `CHECK`	Standard unit: `'KG'`, `'TON'`, `'BAG'`, `'CARTON'`, `'LITRE'`, `'UNIT'`.
`primary_division_id`	`VARCHAR(30)`	`NOT NULL`, `CHECK`	Originating division: `'AGRICORE'`, `'RETAIL'`.
`is_active`	`BOOLEAN`	`DEFAULT TRUE`	Active sellable status.

2.6 `core.financial_account_references`

- **Purpose**: Controlled, non-sensitive reference linking customers and suppliers to VFS financial product records without exposing banking details.
- **Referenced By**: `vfs.wallet_accounts`.

Column Name	Data Type	Constraints	Description / Business Rules
`account_ref_id`	`VARCHAR(30)`	`PRIMARY KEY`	Reference code (e.g., `FAR-WLT-90021`).
`customer_id`	`VARCHAR(30)`	`NOT NULL`, `FK -> core.customers`	Link to customer identity.
`account_type`	`VARCHAR(30)`	`NOT NULL`, `CHECK`	Product type: `'SAVINGS'`, `'WALLET'`, `'LOAN'`.
`account_status`	`VARCHAR(30)`	`DEFAULT 'ACTIVE'`, `CHECK`	Status: `'ACTIVE'`, `'DORMANT'`, `'CLOSED'`.
`opened_date`	`DATE`	`NOT NULL`	Account opening date.

3. Meridian Retail & Consumer Module (`retail` Schema — 7 Entities)

3.1 `retail.stores`

Column Name	Data Type	Constraints	Description
`store_id`	`VARCHAR(30)`	`PRIMARY KEY`	Unique branch identifier (e.g., `STR-LOS-01`).
`location_id`	`VARCHAR(30)`	`NOT NULL`, `FK -> core.locations_sites`	Physical site mapping.
`store_format`	`VARCHAR(50)`	`NOT NULL`, `CHECK`	`'HYPERMARKET'`, `'SUPERMARKET'`, `'EXPRESS'`, `'ONLINE_FULFILLMENT'`.
`opening_date`	`DATE`	`NOT NULL`	Branch opening date.
`manager_employee_id`	`VARCHAR(30)`	`FK -> core.employees`	Assigned store manager.
`property_id`	`VARCHAR(30)`	`FK -> properties.properties`	Premises lease mapping.

3.2 `retail.pos_transactions`

Column Name	Data Type	Constraints	Description
`transaction_id`	`VARCHAR(40)`	`PRIMARY KEY`	Point-of-sale receipt ID.
`store_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> retail.stores`	Store where transaction occurred.
`customer_id`	`VARCHAR(30)`	`FK -> core.customers`	Transacting customer (if loyalty/identified).
`transaction_date`	`TIMESTAMPTZ`	`NOT NULL` , `DEFAULT NOW()`	Date and time of sale.
`total_amount`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Total bill value in local currency (NGN).
`payment_method`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'CASH'` , `'VFS_WALLET'` , `'DEBIT_CARD'` , `'POS'` .

3.3 `retail.transaction_line_items`

Column Name	Data Type	Constraints	Description
`line_item_id`	`VARCHAR(40)`	`PRIMARY KEY`	Unique line item UUID/ID.
`transaction_id`	`VARCHAR(40)`	`NOT NULL` , `FK -> retail.pos_transactions`	Header transaction reference.
`product_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.product_service_catalogue`	Product purchased.
`quantity`	`INT`	`NOT NULL` , `CHECK (> 0)`	Units purchased.
`unit_price`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Unit selling price.

3.4 `retail.inventory_stock_levels`

Column Name	Data Type	Constraints	Description
`stock_id`	`VARCHAR(40)`	`PRIMARY KEY`	Stock record ID.
`store_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> retail.stores`	Store location.
`product_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.product_service_catalogue`	Product SKU.
`quantity_on_hand`	`INT`	`NOT NULL` , `CHECK (>= 0)`	Current stock count.
`last_counted_date`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Last physical audit date.

3.5 `retail.promotions`

Column Name	Data Type	Constraints	Description
`promotion_id`	`VARCHAR(30)`	`PRIMARY KEY`	Promo code ID.
`product_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.product_service_catalogue`	Discounted product.
`discount_type`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'PERCENTAGE'` , `'FIXED_AMOUNT'` , `'BUY_1_GET_1'` .
`discount_value`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Discount rate/amount.
`start_date`	`DATE`	`NOT NULL`	Promo start date.
`end_date`	`DATE`	`NOT NULL` , `CHECK (end_date >= start_date)`	Promo end date.

3.6 `retail.loyalty_accounts`

Column Name	Data Type	Constraints	Description
`loyalty_id`	`VARCHAR(30)`	`PRIMARY KEY`	Loyalty card ID.
`customer_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.customers`	Member customer ID.
`tier`	`VARCHAR(20)`	`DEFAULT 'BRONZE'` , `CHECK`	`'BRONZE'` , `'SILVER'` , `'GOLD'` , `'PLATINUM'` .
`points_balance`	`INT`	`DEFAULT 0` , `CHECK (>= 0)`	Accumulated reward points.
`enrolment_date`	`DATE`	`NOT NULL` , `DEFAULT CURRENT_DATE`	Date joined loyalty program.

3.7 `retail.supplier_deliveries`

Column Name	Data Type	Constraints	Description
`delivery_id`	`VARCHAR(40)`	`PRIMARY KEY`	Inbound delivery manifest ID.
`supplier_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.suppliers_vendors`	Supplier (AgriCore or Wholesale).
`store_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> retail.stores`	Destination retail store.
`expected_date`	`DATE`	`NOT NULL`	Scheduled delivery date.
`received_date`	`DATE`	Nullable	Actual receipt date.
`status`	`VARCHAR(30)`	`DEFAULT 'SCHEDULED'` , `CHECK`	`'SCHEDULED'` , `'RECEIVED'` , `'DELAYED'` , `'SHORTFALL'` , `'CANCELLED'` .

---

4. Veridian Financial Services Module (`vfs` Schema — 6 Entities)

4.1 `vfs.wallet_accounts`

Column Name	Data Type	Constraints	Description
`wallet_id`	`VARCHAR(30)`	`PRIMARY KEY`	Digital wallet account ID.
`customer_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.customers`	Wallet holder.
`account_ref_id`	`VARCHAR(30)`	`FK -> core.financial_account_references`	Core hub account ref.
`balance`	`DECIMAL(18,2)`	`DEFAULT 0.00` , `CHECK (>= 0)`	Current wallet funds (NGN).
`status`	`VARCHAR(30)`	`DEFAULT 'ACTIVE'` , `CHECK`	`'ACTIVE'` , `'BLOCKED'` , `'SUSPENDED'` .

4.2 `vfs.wallet_transactions`

Column Name	Data Type	Constraints	Description
`wallet_txn_id`	`VARCHAR(40)`	`PRIMARY KEY`	Transaction ledger entry ID.
`wallet_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> vfs.wallet_accounts`	Source wallet.
`counterparty_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	`'MERIDIAN_RETAIL_POS'` , `'AGRICORE_PAYOUT'` , `'TRANSFER'` , `'DEPOSIT'` , `'WITHDRAWAL'` .
`amount`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Transaction value.
`transaction_date`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Execution timestamp.

4.3 `vfs.loans`

Column Name	Data Type	Constraints	Description
`loan_id`	`VARCHAR(30)`	`PRIMARY KEY`	Unique credit facility ID.
`borrower_customer_id`	`VARCHAR(30)`	`FK -> core.customers`	Borrower (if individual customer).
`borrower_supplier_id`	`VARCHAR(30)`	`FK -> core.suppliers_vendors`	Borrower (if farmer/supplier).
`principal_amount`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Approved principal loan amount.
`interest_rate`	`DECIMAL(5,2)`	`NOT NULL` , `CHECK (> 0)`	Annual percentage interest rate.
`status`	`VARCHAR(30)`	`DEFAULT 'ACTIVE'` , `CHECK`	`'APPLIED'` , `'DISBURSED'` , `'ACTIVE'` , `'PAID_OFF'` , `'DEFAULTED'` .

4.4 `vfs.loan_repayments`

Column Name	Data Type	Constraints	Description
`repayment_id`	`VARCHAR(40)`	`PRIMARY KEY`	Repayment schedule entry ID.
`loan_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> vfs.loans`	Master loan ID.
`due_date`	`DATE`	`NOT NULL`	Scheduled repayment date.
`amount_due`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Required installment amount.
`paid_amount`	`DECIMAL(18,2)`	`DEFAULT 0.00` , `CHECK (>= 0)`	Actual amount paid.
`paid_date`	`DATE`	Nullable	Payment receipt date.

4.5 `vfs.kyc_records`

Column Name	Data Type	Constraints	Description
`kyc_id`	`VARCHAR(30)`	`PRIMARY KEY`	Compliance verification ID.
`customer_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.customers`	Customer verified.
`verification_level`	`VARCHAR(20)`	`NOT NULL` , `CHECK`	Regulatory tier: `'TIER_1'` , `'TIER_2'` , `'TIER_3'` .
`verified_date`	`DATE`	`NOT NULL` , `DEFAULT CURRENT_DATE`	Approval date.

4.6 `vfs.merchant_settlements`

Column Name	Data Type	Constraints	Description
`settlement_id`	`VARCHAR(40)`	`PRIMARY KEY`	Daily/weekly settlement batch ID.
`division_id`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	Recipient division: `'RETAIL'` , `'LOGISTICS'` , `'AGRICORE'` .
`settlement_date`	`DATE`	`NOT NULL`	Settlement date.
`total_amount`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Total net payout amount.

5. AgriCore Agribusiness Module (`agricore` Schema — 7 Entities)

5.1 `agricore.farms`

Column Name	Data Type	Constraints	Description
`farm_id`	`VARCHAR(30)`	`PRIMARY KEY`	Farm plot ID (e.g., `FRM-KAD-001`).
`supplier_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.suppliers_vendors`	Registered farmer/owner ID.
`location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Geographical location mapping.
`size_hectares`	`DECIMAL(8,2)`	`NOT NULL` , `CHECK (> 0)`	Plot area in hectares.
`primary_crop`	`VARCHAR(50)`	`NOT NULL`	Dominant crop (e.g., `Maize` , `Plantain` , `Cowpea`).

5.2 `agricore.farmers`

Column Name	Data Type	Constraints	Description
`farmer_id`	`VARCHAR(30)`	`PRIMARY KEY`	Farmer profile ID.
`supplier_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.suppliers_vendors`	Shared supplier core entity.
`registration_date`	`DATE`	`NOT NULL`	Date registered with AgriCore.
`cooperative_name`	`VARCHAR(150)`	Nullable	Farmer cooperative association name.

5.3 `agricore.harvest_batches`

Column Name	Data Type	Constraints	Description
`harvest_id`	`VARCHAR(30)`	`PRIMARY KEY`	Harvest batch collection ID.
`farm_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> agricore.farms`	Source farm.
`product_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.product_service_catalogue`	Harvested commodity SKU.
`harvest_date`	`DATE`	`NOT NULL`	Collection date.
`volume_kg`	`DECIMAL(12,2)`	`NOT NULL` , `CHECK (> 0)`	Gross harvested volume (kg).
`field_agent_employee_id`	`VARCHAR(30)`	`FK -> core.employees`	AgriCore field collection agent.

5.4 `agricore.processing_runs`

Column Name	Data Type	Constraints	Description
`run_id`	`VARCHAR(30)`	`PRIMARY KEY`	Facility processing batch ID.
`harvest_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> agricore.harvest_batches`	Input harvest batch.
`facility_location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Processing plant location.
`run_date`	`DATE`	`NOT NULL`	Processing date.
`output_volume_kg`	`DECIMAL(12,2)`	`NOT NULL` , `CHECK (>= 0)`	Net processed output (kg).

5.5 `agricore.quality_grades`

Column Name	Data Type	Constraints	Description
`grade_id`	`VARCHAR(30)`	`PRIMARY KEY`	Quality inspection record ID.
`run_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> agricore.processing_runs`	Processing run inspected.
`grade_level`	`VARCHAR(20)`	`NOT NULL` , `CHECK`	Quality grade: `'GRADE_A'` , `'GRADE_B'` , `'GRADE_C'` , `'REJECTED'` .
`moisture_content`	`DECIMAL(5,2)`	Nullable	Moisture percentage measure.
`inspector_employee_id`	`VARCHAR(30)`	`FK -> core.employees`	QA inspector employee ID.

5.6 `agricore.wholesale_shipments`

Column Name	Data Type	Constraints	Description
`wholesale_id`	`VARCHAR(30)`	`PRIMARY KEY`	Wholesale dispatch ID.
`run_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> agricore.processing_runs`	Source processed batch.
`destination_type`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'MERIDIAN_RETAIL_STORE'` , `'THIRD_PARTY_CLIENT'` .
`destination_id`	`VARCHAR(30)`	`NOT NULL`	Target store/client ID.
`shipment_id`	`VARCHAR(40)`	`FK -> logistics.shipments`	Transport shipment link.

5.7 `agricore.farmer_loans_reference`

Column Name	Data Type	Constraints	Description
`reference_id`	`VARCHAR(30)`	`PRIMARY KEY`	Cross-divisional reference ID.
`farmer_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> agricore.farmers`	AgriCore farmer ID.
`loan_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> vfs.loans`	VFS loan ID.
`visible_summary_status`	`VARCHAR(50)`	`NOT NULL`	Consent-filtered loan summary status.

---

6. Concord Logistics Module (`logistics` Schema — 7 Entities)

6.1 `logistics.vehicles`

Column Name	Data Type	Constraints	Description
`vehicle_id`	`VARCHAR(30)`	`PRIMARY KEY`	Fleet vehicle ID (e.g., `VEH-TRK-012`).
`registration_number`	`VARCHAR(30)`	`UNIQUE` , `NOT NULL`	Official license plate number.
`vehicle_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	`'LAST_MILE_VAN'` , `'HAULAGE_TRUCK'` , `'REFRIGERATED_TRUCK'` .
`capacity_kg`	`INT`	`NOT NULL` , `CHECK (> 0)`	Payload weight limit (kg).
`status`	`VARCHAR(30)`	`DEFAULT 'AVAILABLE'` , `CHECK`	Fleet status: `'AVAILABLE'` , `'IN_TRANSIT'` , `'MAINTENANCE'` .

6.2 `logistics.drivers`

Column Name	Data Type	Constraints	Description
`driver_id`	`VARCHAR(30)`	`PRIMARY KEY`	Driver profile ID.
`employee_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.employees`	Driver employee record.
`licence_number`	`VARCHAR(50)`	`UNIQUE` , `NOT NULL`	Commercial driver license number.
`licence_expiry`	`DATE`	`NOT NULL`	Driver license expiration date.

6.3 `logistics.shipments`

Column Name	Data Type	Constraints	Description
`shipment_id`	`VARCHAR(40)`	`PRIMARY KEY`	Master freight shipment ID.
`origin_location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Pickup location.
`destination_location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Drop-off location.
`client_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	`'INTERNAL_AGRICORE'` , `'INTERNAL_RETAIL'` , `'THIRD_PARTY_COMMERCIAL'` .
`status`	`VARCHAR(30)`	`DEFAULT 'SCHEDULED'` , `CHECK`	`'SCHEDULED'` , `'IN_TRANSIT'` , `'DELIVERED'` , `'DELAYED'` , `'CANCELLED'` .

6.4 `logistics.shipment_legs`

Column Name	Data Type	Constraints	Description
`leg_id`	`VARCHAR(40)`	`PRIMARY KEY`	Multi-leg route segment ID.
`shipment_id`	`VARCHAR(40)`	`NOT NULL` , `FK -> logistics.shipments`	Master shipment ID.
`vehicle_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> logistics.vehicles`	Assigned vehicle.
`driver_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> logistics.drivers`	Assigned driver.
`departure_time`	`TIMESTAMPTZ`	Nullable	Actual departure timestamp.
`arrival_time`	`TIMESTAMPTZ`	Nullable	Actual arrival timestamp.

6.5 `logistics.routes`

Column Name	Data Type	Constraints	Description
`route_id`	`VARCHAR(30)`	`PRIMARY KEY`	Standard transit corridor ID.
`route_name`	`VARCHAR(150)`	`NOT NULL`	Corridor name (e.g., `Kano to Lagos Haulage`).
`origin_location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Origin site.
`destination_location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Destination site.
`distance_km`	`DECIMAL(8,2)`	`NOT NULL` , `CHECK (> 0)`	Route distance in kilometers.

6.6 `logistics.warehouses`

Column Name	Data Type	Constraints	Description
`warehouse_id`	`VARCHAR(30)`	`PRIMARY KEY`	Logistics depot ID.
`location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Physical location ID.
`capacity_units`	`INT`	`NOT NULL` , `CHECK (> 0)`	Pallet/unit storage capacity.
`leased_from_property_id`	`VARCHAR(30)`	`FK -> properties.properties`	Veridian Properties lease reference.

6.7 `logistics.maintenance_logs`

Column Name	Data Type	Constraints	Description
`maintenance_id`	`VARCHAR(40)`	`PRIMARY KEY`	Fleet repair log entry ID.
`vehicle_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> logistics.vehicles`	Serviced vehicle.
`service_date`	`DATE`	`NOT NULL`	Maintenance date.
`cost`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (>= 0)`	Repair cost in NGN.
`description`	`TEXT`	Nullable	Maintenance work summary.

7. Veridian Properties Module (`properties` Schema — 7 Entities)

7.1 `properties.properties`

Column Name	Data Type	Constraints	Description
`property_id`	`VARCHAR(30)`	`PRIMARY KEY`	Real estate asset ID (e.g., `PROP-LOS-004`).
`location_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> core.locations_sites`	Site location mapping.
`property_type`	`VARCHAR(50)`	`NOT NULL` , `CHECK`	`'RETAIL_STORE_PREMISES'` , `'LOGISTICS_WAREHOUSE'` , `'AGRI_FACILITY'` , `'COMMERCIAL_OFFICE'` .
`size_sqm`	`DECIMAL(10,2)`	`NOT NULL` , `CHECK (> 0)`	Total floor area in square meters.
`ownership_status`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'OWNED_FREEHOLD'` , `'LEASED_FROM_LANDLORD'` .

7.2 `properties.tenants`

Column Name	Data Type	Constraints	Description
`tenant_id`	`VARCHAR(30)`	`PRIMARY KEY`	Occupant tenant ID.
`tenant_type`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'INTERNAL_DIVISION'` , `'EXTERNAL_COMMERCIAL_TENANT'` .
`division_id`	`VARCHAR(30)`	Nullable, `CHECK`	Internal division: `'RETAIL'` , `'LOGISTICS'` , `'AGRICORE'` , `'VFS'` .
`external_tenant_name`	`VARCHAR(150)`	Nullable	Commercial client company name.

7.3 `properties.leases`

Column Name	Data Type	Constraints	Description
`lease_id`	`VARCHAR(30)`	`PRIMARY KEY`	Property lease contract ID.
`property_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.properties`	Leased property asset.
`tenant_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.tenants`	Tenant contract holder.
`start_date`	`DATE`	`NOT NULL`	Lease commencement date.
`end_date`	`DATE`	`NOT NULL`	Lease expiration date (`CHECK end_date > start_date`).
`monthly_rent`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Monthly rental fee in NGN.

7.4 `properties.maintenance_requests`

Column Name	Data Type	Constraints	Description
`request_id`	`VARCHAR(40)`	`PRIMARY KEY`	Property defect/repair request ID.
`property_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.properties`	Target property asset.
`requested_date`	`DATE`	`NOT NULL` , `DEFAULT CURRENT_DATE`	Date ticket opened.
`category`	`VARCHAR(50)`	`NOT NULL`	Defect category (e.g., `Roofing` , `HVAC` , `Electrical`).
`status`	`VARCHAR(30)`	`DEFAULT 'OPEN'` , `CHECK`	`'OPEN'` , `'IN_PROGRESS'` , `'RESOLVED'` .
`resolved_date`	`DATE`	Nullable	Ticket resolution date.

7.5 `properties.property_valuations`

Column Name	Data Type	Constraints	Description
`valuation_id`	`VARCHAR(40)`	`PRIMARY KEY`	Portfolio audit valuation ID.
`property_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.properties`	Valued asset ID.
`valuation_date`	`DATE`	`NOT NULL`	Valuation audit date.
`assessed_value`	`DECIMAL(18,2)`	`NOT NULL` , `CHECK (> 0)`	Assessed market value in NGN.

7.6 `properties.utility_accounts`

Column Name	Data Type	Constraints	Description
`utility_id`	`VARCHAR(30)`	`PRIMARY KEY`	Utility account mapping ID.
`property_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.properties`	Property premises.
`utility_type`	`VARCHAR(30)`	`NOT NULL` , `CHECK`	`'ELECTRICITY'` , `'WATER'` , `'WASTE_MANAGEMENT'` .
`provider_name`	`VARCHAR(100)`	`NOT NULL`	Utility provider name (e.g., `IKEDC` , `LWC`).
`account_number`	`VARCHAR(50)`	`NOT NULL`	Meter / account number.

7.7 `properties.facility_assets`

Column Name	Data Type	Constraints	Description
`asset_id`	`VARCHAR(40)`	`PRIMARY KEY`	Capital equipment ID.
`property_id`	`VARCHAR(30)`	`NOT NULL` , `FK -> properties.properties`	Property location.
`asset_type`	`VARCHAR(50)`	`NOT NULL`	Asset classification (e.g., `Generator 500kVA` , `Chiller Unit`).
`installed_date`	`DATE`	`NOT NULL`	Installation date.
`condition_rating`	`VARCHAR(30)`	`CHECK`	Condition: `'EXCELLENT'` , `'GOOD'` , `'FAIR'` , `'NEEDS_REPLACEMENT'` .

---

8. Summary of Key Analytical Views Exposed for BI

View Name	Schema	Base Tables Joined	Key Purpose / Scenario Addressed
`v_agricore_retail_supply_alert`	`public`	`retail.supplier_deliveries` , `agricore.harvest_batches` , `core.product_service_catalogue` , `retail.stores`	**Scenario 1 (Ngozi)**: Provides store managers advance supply shortfall warnings.
`v_farmer_credit_assessment`	`public`	`agricore.farmers` , `agricore.harvest_batches` , `vfs.loans` , `core.customers` , `core.suppliers_vendors`	**Scenario 2 (Chinedu)**: Consent-based multi-year farmer harvest history for credit scoring.
`v_logistics_warehouse_leases`	`public`	`logistics.warehouses` , `properties.properties` , `properties.leases` , `properties.tenants`	**Scenario 3 (Funmi)**: Real-time warehouse lease status and double-billing prevention.
`v_consolidated_group_revenue`	`public`	`retail.pos_transactions` , `logistics.shipments` , `vfs.wallet_transactions` , `agricore.wholesale_shipments` , `properties.leases`	**Scenario 4 (Adaeze)**: Same-day cross-divisional revenue consolidation for executive oversight.

Architected and Structured by Samuel Enyi, Lead Database Architect & SQL Engineer.

Project Concord: Power BI Dashboard Build Guide

Ref: VCH-PC-2026-POWERBI-GUIDE | **Version**: 3.0 (Unified Design System)
Author: Samuel Enyi | **Defense Date**: 20 August 2026
Developers: Christian Nwobodo & Richmond Akara-Nwaogu

PART A: UNDERSTAND THE PROBLEM FIRST

Read this before touching Power BI. Your dashboards must answer specific business questions.
If you do not know the questions, you cannot build the right answers.

The 4 Business Problems Your Dashboards Solve

#	Problem	Who It Hurts	What Goes Wrong
1	**21-Day Revenue Delay**	CEO Adaeze	After month-end, all 5 divisions email separate spreadsheets. Finance manually combines them. It takes 21 days. 15.3 billion naira wasted yearly.
2	**Invisible Farmer**	VFS Loan Officer Chinedu	A farmer with 6 years of reliable AgriCore deliveries applies for a loan. Chinedu only sees 2 years of banking history. VFS and AgriCore never shared data.
3	**Double-Booked Warehouse**	Logistics Dispatcher Funmi	Funmi sends a truck to a depot bay. When it arrives, the bay is already leased to someone else. Properties changed it by email and Funmi never saw it.
4	**Empty Shelf**	Store Manager Ngozi	AgriCore farms lose 40% of their harvest to bad weather. Nobody tells Ngozi. Her Enugu store runs out of stock 4 days later with no warning.

How the Database Fixed This

Samuel built 40 tables across 6 database schemas (`core` , `retail` , `vfs` , `agricore` , `logistics` , `properties`).
All divisions now share **6 common master tables** in the `core` schema:

Shared Table	What It Connects
core.customers	Same customer ID used by both Retail and VFS
core.suppliers_vendors	Same supplier ID used by both AgriCore and Retail
core.employees	All staff across all 5 divisions
core.locations_sites	All stores, warehouses, and farms share one address system
core.product_service_catalogue	Produce harvested by AgriCore = products sold in Meridian Retail
core.financial_account_references	Non-sensitive bridge so other divisions can see VFS account existence

These cross-division queries are pre-built as **4 SQL Views** in the database:

SQL View	Problem It Solves
`core.v_executive_consolidated_revenue`	Adaeze: Group revenue in 3 seconds, not 21 days
`vfs.v_farmer_credit_evaluation`	Chinedu: See 6 years of a farmer harvest history
`logistics.v_warehouse_lease_utilization`	Funmi: Live lease expiry status per depot
`retail.v_agricore_retail_supply_alert`	Ngozi: Harvest shortfall alert 4 days in advance

Where Your Dashboard Fits

- **Database + Views** = solved the data problem (numbers are now connected across all divisions)
- **Your Power BI Dashboard** = solves the human problem (makes the data readable for business users)

> The dashboards are the final layer. Without them, only engineers can access this platform.

PART B: UNIFIED VCH CORPORATE COLOR SYSTEM (BOTH DEVELOPERS MUST FOLLOW)

To keep the dashboards clean, consistent, and professional, **both Christian and Richmond will use the exact same corporate color system across all 4 pages**.
Avoid using too many random colors. Keep standard charts neutral, and only use alert colors (Red/Amber/Green) when highlighting specific operational statuses or warnings.

Master Palette Table

Element	Color Name	Hex Code	Usage
Page Background	Corporate Slate	`#0F172A`	Background for ALL dashboard pages
Card / Container Background	Dark Slate Container	`#1E293B`	Background for ALL cards, visual containers, and tables (high contrast)
Primary Accent & Headers	Sky Cyan	`#38BDF8`	Titles, primary bar chart bars, line chart active series, card borders
Primary Text	Crisp White	`#F8FAFC`	All main text, numbers, and KPI values (100% readable)
Secondary Text	Muted Grey	`#94A3B8`	Subtitles, axis labels, table column headers
Status: Good / Healthy / Active	Emerald Green	`#22C55E`	Active leases, healthy stock (>200 units), paid loans, NDPA consent TRUE
Status: Warning / Expiring / Low	Amber Orange	`#F59E0B`	Low stock (50-200 units), lease expiring in 90 days, active loan
Status: Critical / Expired / Default	Coral Red	`#EF4444`	Stock critical (<50 units), expired lease, defaulted loan, delayed shipment

PART C: CONNECT TO THE DATABASE (BOTH DEVELOPERS DO THIS)

Step 1 — Install the Driver

1. Go to https://www.npgsql.org/ and download the Npgsql PostgreSQL driver
2. Install it, then restart Power BI Desktop

Step 2 — Open a New Connection

1. Open **Power BI Desktop**
2. Click **Home** ribbon → **Get Data**
3. Search for `PostgreSQL` → select **PostgreSQL database** → click **Connect**

Step 3 — Enter Server Details

- **Server**: `aws-1-eu-west-2.pooler.supabase.com:6543`
- **Database**: `postgres`
- **Data Connectivity mode**: `Import`

Click **OK**.

Step 4 — Enter Login Credentials

When the login window appears, click the **Database** tab (not Windows):
- **User name**: `postgres.hraipqaksitkchkhhdye`
- **Password**: `Fi4nMf2MXD668s3x`

Click **Connect**.

Step 5 — Select Your Data

1. The **Navigator panel** opens showing all schemas
2. Expand the schema folder (e.g., `core` , `retail` , `vfs`)
3. Tick the checkbox next to each view or table you need
4. Click **Load** at the bottom right

PART D: CHRISTIAN NWOBODO'S DASHBOARD

Dashboard File Name: `VCH_Executive_Retail_Intelligence_Dashboard.pbix`

Your job — answer 2 questions:
- **Page 1**: What is VCH group revenue across all divisions right now? *(Adaeze)*
- **Page 2**: Which stores are about to run out of stock? *(Ngozi)*

D1 — Load These Data Sources

Primary Views:

What to Load	Where in Navigator	Key Columns
`core.v_executive_consolidated_revenue`	Expand `core` then tick the view	`division`, `total_revenue_ngn`, `transaction_count`, `last_transaction_date`
`retail.v_agricore_retail_supply_alert`	Expand `retail` then tick the view	`store_city`, `product_name`, `retail_stock_on_hand`, `recent_harvest_volume_kg`, `delivery_status`

Supporting Tables:

Table	Schema	Used For
`pos_transactions`	retail	Monthly sales trend chart
`stores`	retail	Store format filter
`inventory_stock_levels`	retail	Stock quantity visuals
`product_service_catalogue`	core	Product category filter
`locations_sites`	core	City and country filter

D2 — Build Page 1: Executive Revenue Command Centre

Style: Professional executive cockpit. Dark Slate background with high-contrast card containers.

Apply Colors from Master Palette (Part B):
- Page background: `#0F172A`
- Card background: `#1E293B`
- Main Accent: `#38BDF8`
- Text: `#F8FAFC` (Primary), `#94A3B8` (Secondary)

Build top to bottom in 5 rows:

ROW 1 — Header
1. Insert → Text Box → stretch full width at top
2. Title: `VCH Group Executive Performance Dashboard` (Color: `#38BDF8`, Bold)
3. Subtitle: `Live Cross-Divisional Revenue Intelligence | Powered by Project Concord` (Color: `#94A3B8`)
4. Right Text Box: `Last Refreshed: [today's date]`

ROW 2 — 4 KPI Cards (equal width, side by side)

For each card: Card Visual → set Background Color to `#1E293B`.

Card	Field	Format
Total Group Revenue	`total_revenue_ngn` (SUM) from revenue view	Billions with Naira symbol (`₦`)
Total Transactions	`transaction_count` (SUM) from revenue view	Number with comma separators
Top Division	Create DAX measure (see D4)	Text string
Lowest Division	Create DAX measure (see D4)	Text string

ROW 3 — 2 Charts (side by side)

LEFT (60% of row width) — Revenue by Division
- Visual: **Clustered Bar Chart**
- X-axis: `division` from `v_executive_consolidated_revenue`
- Y-axis: `total_revenue_ngn` (SUM)
- Bar color: `#38BDF8` (Sky Cyan for consistency)
- Sort bars descending (highest revenue on top)
- Data Labels: ON (Color: `#F8FAFC`)

RIGHT (40% of row width) — Transactions by Division
- Visual: **Donut Chart**
- Values: `transaction_count`
- Legend: `division`
- Colors: Monochromatic blues/cyans (`#38BDF8` , `#0284C7` , `#0369A1` , `#075985`)

ROW 4 — Monthly Sales Trend (full width)
- Visual: **Line Chart**
- X-axis: `transaction_date` from `pos_transactions` → group by **Month**
- Y-axis: SUM of `total_amount`
- Legend: `payment_method`
- Analytics pane: add an **Average Line** (`#94A3B8` dashed)
- Title: `Meridian Retail Monthly Sales Trend by Payment Method`

ROW 5 — Summary Table (full width)
- Visual: **Table**
- Source: `v_executive_consolidated_revenue`
- Columns: `division` , `total_revenue_ngn` , `transaction_count` , `last_transaction_date`
- Table background: `#1E293B` , Header font: `#38BDF8`
- Conditional formatting on `total_revenue_ngn` :
- Top value → `#22C55E` (Green)
- Middle values → `#38BDF8` (Cyan)
- Lowest value → `#EF4444` (Red)

D3 — Build Page 2: Retail Supply Alert System

Style: Operational alert view. Same slate background, traffic-light status colors for stock alerts only.

Build top to bottom in 5 rows:

ROW 1 — Header
- Title: `Meridian Retail — AgriCore Supply Chain Alert System` (`#38BDF8`)
- Subtitle: `Real-Time Harvest-to-Shelf Supply Monitoring` (`#94A3B8`)

ROW 2 — 3 KPI Cards

Card	How to Build
Total Stores Monitored	Card → `store_id` from supply alert view → Count Distinct
Products with Stock Risk	Use DAX measure `Stock Risk Count` (see D4) — Card BG `#1E293B` , Number color `#EF4444`
Active Deliveries	Card → `delivery_status` → count excluding `NO_DELIVERY_LOGGED`

ROW 3 — Alert Status Table (full width)
- Visual: **Table**
- Source: `retail.v_agricore_retail_supply_alert`
- Columns: `store_city` , `product_name` , `retail_stock_on_hand` , `recent_harvest_volume_kg` , `delivery_status`
- Conditional formatting on `retail_stock_on_hand` (Background color):
- Below 50 → Red `#EF4444` (CRITICAL)
- 50 to 200 → Amber `#F59E0B` (LOW)
- Above 200 → Green `#22C55E` (HEALTHY)

ROW 4 — 2 Charts

LEFT — Stock by Store
- Visual: **Horizontal Bar Chart**
- Y-axis: `store_city`

- X-axis: `retail_stock_on_hand`
- Sort ascending (lowest stock at top)
- Bar color: `#38BDF8`

RIGHT — Harvest Volume by Product
- Visual: **Treemap**
- Category: `product_name`
- Values: `recent_harvest_volume_kg`
- Color gradient: Slate blue to Sky Cyan (`#1E3A5F` to `#38BDF8`)

ROW 5 — Slicers
- 3 slicers along bottom: `store_city` , `product_name` , `delivery_status`
- Slicer container background: `#1E293B`

D4 — DAX Measures for Christian

```
Total Group Revenue = SUM(v_executive_consolidated_revenue[total_revenue_ngn])

Total Transactions = SUM(v_executive_consolidated_revenue[transaction_count])

Stock Risk Count =
    CALCULATE(
        COUNTROWS(v_agricore_retail_supply_alert),
        v_agricore_retail_supply_alert[retail_stock_on_hand] < 100
    )
```

PART E: RICHMOND AKARA-NWAOGU'S DASHBOARD

Dashboard File Name: `VCH_Credit_Risk_Depot_Utilization_Dashboard.pbix`

Your job — answer 2 questions:
- **Page 1**: Is this farmer creditworthy based on their full harvest history? *(Chinedu)*
- **Page 2**: Which depots have expired or expiring leases? *(Funmi)*

E1 — Load These Data Sources

Primary Views:

What to Load	Where in Navigator	Key Columns
`vfs.v_farmer_credit_evaluation`	Expand `vfs` then tick the view	`farmer_name` , `farm_id` , `primary_crop` , `total_harvest_batches` , `cumulative_harvest_volume_kg` , `first_harvest_date` , `last_harvest_date` , `consent_vfs_credit_sharing`
`logistics.v_warehouse_lease_utilization`	Expand `logistics` then tick the view	`depot_name` , `city` , `total_pallet_capacity` , `ownership_status` , `monthly_rent` , `lease_start` , `lease_end` , `lease_status`

Supporting Tables:

Table	Schema	Used For
`loans`	vfs	Loan status and value KPI cards
`loan_repayments`	vfs	Repayment trend chart
`harvest_batches`	agricore	Harvest trend over time
`warehouses`	logistics	Depot capacity bar chart
`shipments`	logistics	Shipment status chart
`properties`	properties	Property ownership pie chart

E2 — Build Page 1: Farmer Credit Risk Intelligence Panel

Style: Credit bureau panel. Exactly same slate background (`#0F172A`) and container background (`#1E293B`) as Christian's dashboard.

Build top to bottom in 6 rows:

ROW 1 — Header
- Title: `VFS Microfinance — Farmer Credit Risk Intelligence Panel` (`#38BDF8`)
- Subtitle: `AgriCore-Integrated Cross-Divisional Credit Evaluation System` (`#94A3B8`)

ROW 2 — 4 KPI Cards (Background: `#1E293B`)

Card	How to Build
Total Farmers Evaluated	Use DAX measure `Total Farmers` (see E4)
Total Active Loans	`status` from `loans` filtered to ACTIVE → Count
Total Loan Portfolio	SUM of `principal_amount` from `loans`
Avg Harvest Per Farmer	Use DAX measure `Avg Harvest Per Farmer` (see E4)

ROW 3 — 2 Charts

LEFT — Top 10 Farmers by Harvest Volume
- Visual: **Horizontal Bar Chart**
- Y-axis: `farmer_name` from `v_farmer_credit_evaluation`
- X-axis: `cumulative_harvest_volume_kg`
- Bar color: `#38BDF8` (Sky Cyan)
- Top N filter: Top 10 by volume
- Title: `Top 10 Farmers — 6-Year Cumulative Supply Volume`

RIGHT — Harvest Mix by Crop
- Visual: **Donut Chart**
- Values: `cumulative_harvest_volume_kg`
- Legend: `primary_crop`
- Color palette: Monochromatic blues/cyans (`#38BDF8` , `#0284C7` , `#0369A1` , `#075985`)

ROW 4 — Harvest Trend (full width)
- Visual: **Line Chart**
- Source: `agricore.harvest_batches`
- X-axis: `harvest_date` → group by **Month**
- Y-axis: SUM of `volume_kg`
- Legend: `primary_crop` (top 5 crops)
- Title: `AgriCore Monthly Harvest Volume Trend`

ROW 5 — Farmer Credit Scorecard Table (full width)
- Visual: **Table**
- Source: `vfs.v_farmer_credit_evaluation`
- Columns: `farmer_name` , `farm_id` , `primary_crop` , `total_harvest_batches` , `cumulative_harvest_volume_kg` , `first_harvest_date` , `last_harvest_date` , `consent_vfs_credit_sharing`
- Sort by `cumulative_harvest_volume_kg` descending
- Conditional format on `consent_vfs_credit_sharing` :

- TRUE → Green cell (`#22C55E`) — Consent Granted
- FALSE → Muted grey cell (`#64748B`) — Consent Restricted (NDPA)

ROW 6 — 2 Charts

LEFT — Loan Status Breakdown
- Visual: **Clustered Bar Chart**
- Source: `vfs.loans`
- X-axis: `status`
- Y-axis: Count of `loan_id`
- Bar colors: PAID_OFF = `#22C55E` (Green), ACTIVE = `#F59E0B` (Amber), DEFAULTED = `#EF4444` (Red)

RIGHT — Repayment Performance
- Visual: **Stacked Bar Chart**
- Source: `vfs.loan_repayments`
- X-axis: `due_date` → group by Month
- Y-axis stacks: `amount_due` (`#64748B` grey) then `paid_amount` (`#22C55E` green) on top

---

E3 — Build Page 2: Warehouse Lease & Depot Utilization Panel

Style: Depot operations panel. Same slate background (`#0F172A`) and containers (`#1E293B`).

Build top to bottom in 6 rows:

ROW 1 — Header
- Title: `Concord Logistics — Warehouse Lease & Depot Capacity Intelligence` (`#38BDF8`)
- Subtitle: `Real-Time Property Lease Status & Bay Availability Monitoring` (`#94A3B8`)

ROW 2 — 4 KPI Cards

Card	How to Build	Number Color
Total Depots	COUNTROWS of `v_warehouse_lease_utilization`	`#F8FAFC` (White)
Active Leases	Use DAX `Active Leases` (see E4)	`#22C55E` (Green)
Expiring in 90 Days	Use DAX `Expiring Leases` (see E4)	`#F59E0B` (Amber)
Expired Leases	Filter `lease_status = EXPIRED_LEASE` then Count	`#EF4444` (Red)

ROW 3 — Depot Lease Status Table (full width)
- Visual: **Table**
- Source: `logistics.v_warehouse_lease_utilization`
- Columns: `depot_name` , `city` , `total_pallet_capacity` , `ownership_status` , `monthly_rent` , `lease_start` , `lease_end` , `lease_status`
- Conditional format on `lease_status` (Background color):
- `ACTIVE_LEASE` → Green `#22C55E`
- `RENEWAL_DUE_SOON` → Amber `#F59E0B`
- `EXPIRED_LEASE` → Red `#EF4444`

ROW 4 — 2 Charts

LEFT — Lease Status Distribution
- Visual: **Donut Chart**
- Values: Count of warehouses
- Legend: `lease_status`
- Segment colors: ACTIVE = `#22C55E` , RENEWAL_DUE_SOON = `#F59E0B` , EXPIRED = `#EF4444`

RIGHT — Monthly Rent by City
- Visual: **Bar Chart**
- X-axis: `city`
- Y-axis: SUM of `monthly_rent`
- Bar color: `#38BDF8`

ROW 5 — Depot Capacity (full width)
- Visual: **Horizontal Bar Chart**
- Source: `logistics.warehouses`
- Y-axis: `warehouse_id`

- X-axis: `capacity_units`
- Bar color: `#38BDF8`

ROW 6 — 2 Charts

LEFT — Shipment Status
- Visual: **Clustered Bar Chart**
- Source: `logistics.shipments`
- X-axis: `status`
- Y-axis: Count of shipments
- Bar colors: DELIVERED = `#22C55E` , IN_TRANSIT = `#F59E0B` , DELAYED/CANCELLED = `#EF4444`

RIGHT — Property Ownership Mix
- Visual: **Pie Chart**
- Source: `properties.properties`
- Values: Count of properties
- Legend: `ownership_status`
- Colors: `#38BDF8` (Owned) and `#0284C7` (Leased)

E4 — DAX Measures for Richmond

```
Total Farmers = COUNTROWS(v_farmer_credit_evaluation)

Avg Harvest Per Farmer = AVERAGE(v_farmer_credit_evaluation[cumulative_harvest_volume_kg])

Active Loans = CALCULATE(COUNTROWS(loans), loans[status] = "ACTIVE")

Defaulted Loans = CALCULATE(COUNTROWS(loans), loans[status] = "DEFAULTED")

NPL Ratio = DIVIDE([Defaulted Loans], [Active Loans] + [Defaulted Loans], 0)

Total Loan Portfolio = SUM(loans[principal_amount])

Repayment Rate = DIVIDE(SUM(loan_repayments[paid_amount]), SUM(loan_repayments[amount_due]), 0)

Active Leases =
    CALCULATE(
        COUNTROWS(v_warehouse_lease_utilization),
        v_warehouse_lease_utilization[lease_status] = "ACTIVE_LEASE"
    )

Expiring Leases =
    CALCULATE(
        COUNTROWS(v_warehouse_lease_utilization),
        v_warehouse_lease_utilization[lease_status] = "RENEWAL_DUE_SOON"
    )
```

PART F: FINAL REQUIREMENTS (BOTH DEVELOPERS)

Add Slicers to Every Page

1. **Date Range Slicer**: Relative → Last 30 Days / Last 90 Days / Year to Date / All Time
2. **Country Slicer**: `country_code` from `core.locations_sites` → filter by NG, GH, KE
3. **Division Slicer** *(Christian only, executive page only)*

Slicer background: `#1E293B` , Header text: `#38BDF8` .

Create 3 Bookmarks Per Report (View > Bookmarks > Add Bookmark)

Bookmark Name	Before Saving, Hide These
Executive Summary	All charts and tables — show KPI cards only
Full Detail View	Nothing — show everything
Alert Mode	Filter to show only RED and AMBER rows

Add Page Navigation Buttons

Insert → Buttons → Page Navigator → place at top/bottom of every page.

PART G: DEFENSE PRESENTATION

Before You Start Presenting

1. Save your file (File → Save As)
2. Press **F11** to go full-screen
3. Go to **View** → **Page View** → **Fit to Page**
4. Disable Auto Refresh (Home → uncheck scheduled refresh)

What to Say for Each Page

Christian — Page 1 (Executive Revenue):

> "This page answers CEO Adaeze's question: what is VCH's total group revenue right now?
> Before Project Concord, that answer took 21 days of manual spreadsheet work.
> This dashboard pulls from 5 live divisions in under 3 seconds."

Christian — Page 2 (Supply Alert):

> "This page tells Ngozi which stores are at risk of running out of stock
> before it actually happens, using AgriCore's live harvest data.
> Before Project Concord, Retail and AgriCore shared zero data."

Richmond — Page 1 (Farmer Credit):

> "This page gives Loan Officer Chinedu a 6-year AgriCore harvest history for any farmer —
> a data source that simply did not exist for VFS before Project Concord.
> Data only shows for farmers who have given NDPA consent, enforced at the database level."

Richmond — Page 2 (Depot Lease):

> "This page shows Funmi which depots have expired leases and which are due for renewal.
> Red means expired. Amber means expiring within 90 days.
> The double-booking problem is now impossible to miss."

PART H: COMPLETION CHECKLISTS

Christian — tick as you complete:
- [ ] Connected to Supabase in Power BI Desktop
- [ ] Loaded `core.v_executive_consolidated_revenue`
- [ ] Loaded `retail.v_agricore_retail_supply_alert`
- [ ] Loaded `retail.pos_transactions` , `retail.inventory_stock_levels` , `core.locations_sites`
- [ ] Applied Unified Palette: Background `#0F172A` , Cards `#1E293B` , Accent `#38BDF8`
- [ ] Created 3 DAX measures (Total Group Revenue, Total Transactions, Stock Risk Count)
- [ ] Built Page 1 — Executive Revenue Command Centre (5 rows complete)
- [ ] Built Page 2 — Retail Supply Alert System (5 rows complete)
- [ ] Added Date, Country, and Division slicers on all pages

- [ ] Created 3 bookmarks and page navigation buttons
- [ ] Saved as `VCH_Executive_Retail_Intelligence_Dashboard.pbix`

Richmond — tick as you complete:
- [ ] Connected to Supabase in Power BI Desktop
- [ ] Loaded `vfs.v_farmer_credit_evaluation`
- [ ] Loaded `logistics.v_warehouse_lease_utilization`
- [ ] Loaded `vfs.loans` , `vfs.loan_repayments` , `agricore.harvest_batches`
- [ ] Loaded `logistics.warehouses` , `logistics.shipments` , `properties.properties`
- [ ] Applied Unified Palette: Background `#0F172A` , Cards `#1E293B` , Accent `#38BDF8`
- [ ] Created all 9 DAX measures
- [ ] Built Page 1 — Farmer Credit Risk Intelligence Panel (6 rows complete)
- [ ] Built Page 2 — Warehouse Lease & Depot Utilization Panel (6 rows complete)
- [ ] Added Date and Country slicers on all pages
- [ ] Created 3 bookmarks and page navigation buttons
- [ ] Saved as `VCH_Credit_Risk_Depot_Utilization_Dashboard.pbix`

Document v3.0 | Samuel Enyi — Lead Database Architect & Technical Director | Project Concord | July 2026

Phase 5: Project Execution Chronicle (Weeks 1–4)

Week 1 (14–20 July 2026) — Requirements & Architecture Design

- Israel John (BA Lead) authored the complete Business Requirements Document (BRD) covering all 4 stakeholder scenarios, divisional business rules, KPI definitions, and regulatory constraints.
- Samuel Enyi (Lead Architect) designed the Core Hub & 5 Divisional Module ERD, established the 6-schema PostgreSQL architecture, and confirmed shared entity foreign key design patterns.
- Ruth Thompson (Data Engineering Lead) studied the Mockaroo platform and prepared all synthetic data generation templates across all 19 Core & Divisional tables.

Week 2 (21–27 July 2026) — Schema Build, Data Seeding & Security Implementation (ON TRACK)

- Samuel Enyi authored `01_schema_ddl.sql` — implementing all **43 production tables** across 6 logical schemas (`core` , `retail` , `vfs` , `agricore` , `logistics` , `properties`) with strict PK/FK constraints and CHECK validation rules.
- Samuel Enyi authored `02_security_rls.sql` — enabling PostgreSQL Row Level Security (RLS) on VFS banking tables and configuring 6 role-based access policies (`vfs_ops` , `retail_ops` , `logistics_ops` , `agricore_ops` , `properties_ops` , `group_executive`) for NDPA and Central Bank compliance.
- Samuel Enyi authored `03_analytics_views.sql` — building 4 curated analytical views (`v_agricore_retail_supply_alert` , `v_farmer_credit_evaluation` , `v_warehouse_lease_utilization` , `v_executive_consolidated_revenue`) each directly solving one of the 4 stakeholder scenarios from the client brief.
- Ruth Thompson generated and delivered all synthetic CSV datasets across all tables using Mockaroo.
- Samuel Enyi executed Python cleaning scripts (`clean_core_csvs.py` , `clean_divisional_csvs.py`) to align all CSVs with strict PostgreSQL schema constraints, then provisioned and seeded the production Supabase PostgreSQL database with **950,125 total synthetic records** across all tables.

Week 3 (28 July – 3 August 2026) — CONTINGENCY HOLD (Medical Pause)

- **Sprint Status: PAUSED — Full Team Medical Hold**
- Both Lead BI Developer (Christian Nwobodo) and Associate BI Developer (Richmond Akara-Nwaogu) fell ill during Week 3, creating a critical path dependency bottleneck. All Power BI dashboard construction, BigQuery ELT integration, and dashboard testing were dependent on their availability.
- Project Manager Israel John formally enacted a sprint pause for Week 3. No new deliverables were produced during this period.
- All Week 3 tasks (Power BI connection, dashboard pages, ELT pipeline, EXPLAIN ANALYZE tuning, and defense preparation) were rescheduled and consolidated into a final Week 4 execution sprint.

Week 4 (4–20 August 2026) — Final Execution Sprint & Defense Preparation (COMPLETED)

- Samuel Enyi authored and executed the `elt_supabase_to_bigquery.py` Python ELT pipeline, automating the scheduled extraction of all 4 analytical SQL views from Supabase PostgreSQL and loading them into Google BigQuery as queryable tables for Power BI consumption.
- Christian Nwobodo and Richmond Akara-Nwaogu connected Power BI Desktop to the BigQuery dataset and constructed the complete 4-page Power BI Dashboard (`PROJECT_CONCORD_POWERBI_DASBOARD.pbix`), covering all 4 stakeholder scenarios with live slicers, KPI cards, trend charts, and drill-down tables.
- Samuel Enyi authored the complete Enterprise Data Dictionary (covering all 43 entities), the Lead Architect SQL Guide (with EXPLAIN ANALYZE optimization notes), and the Master Execution Plan.
- Ruth Thompson compiled the Mockaroo Complete Table & Column Spec, the Synthetic Data Generation Spec, and completed final QA validation of dashboard numbers against source CSV records.
- Israel John led the team's defense preparation, confirmed speaking order, and finalized the 4-slide defense deck addressed to Group CEO Adaeze Okonji-Bello and CDTO Tobenna Iweka.
- Final submission package assembled in `07_SQL_PROJECT_CONCORD_SUBMISSION/` and committed to GitHub repository: https://github.com/Coding-Samuel/PROJECT-CONCORD.

Phase 6: Conclusion & Final Governance Recommendations

Project Concord successfully resolves the 20-year data fragmentation across Veridian Consolidated Holdings by implementing a robust, well-governed operational Core Hub surrounded by 5 divisional modules on PostgreSQL (Supabase), connected via an automated ELT pipeline to Google BigQuery, and surfaced through 4 interactive Power BI dashboards directly answering each of the 4 client stakeholder scenarios.

Core System Accomplishments:

1. **950,125 Synthetic Records Seeded**: Full dataset across all 43 tables representing realistic cross-divisional customer, farmer, transaction, and warehouse data.
2. **Unified Core Hub**: 6 shared master entities ensuring single-source-of-truth customer, employee, supplier, location, product, and account reference data across all 5 divisions.
3. **Strict Regulatory Isolation**: Native Row-Level Security (RLS) enforcement protecting Central Bank regulated VFS financial records and enforcing customer consent flags (`consent_vfs_credit_sharing`) under Nigerian Data Protection Act (NDPA).
4. **OLTP/OLAP Separation**: Live transactional integrity on Supabase with zero analytical query lag on BigQuery's columnar warehouse.
5. **Actionable Executive Intelligence**: 4 targeted stakeholder scenario dashboards delivering immediate clarity to store managers (Ngozi), loan officers (Chinedu), logistics dispatchers (Funmi), and the Group CEO (Adaeze).

Post-Submission Governance Recommendations:

- Implement a scheduled daily ELT trigger on Google Cloud Scheduler to keep BigQuery views current.
- Add `overdraft_limit` column to `vfs.wallet_accounts` to fully implement CBN overdraft-aware balance constraint.
- Expand `kyc_records` to include biometric verification tier (Tier 4) as CBN regulations evolve.

Report compiled and certified by Samuel Enyi, Lead Database Architect & SQL Engineer.
Project Concord — Veracity Vanguard Cohort | Veridian Financial Services Division